

EE5203 L11 Study Sheet

I2C and SPI - Basri Erdogan

Essential question: A UART assumes a rate, reaches one place, and never confirms anything. Add a clock wire and all three change. What exactly do you gain, and what do you pay?

What you should be able to do

- Explain what a clock line buys over the UART's assumed rate.
- Read an I2C transaction: START, address, R/W, ACK, data, STOP.
- Convert between 7-bit and HAL's 8-bit address form, both directions.
- Explain open drain and why pull-ups are mandatory.
- State CPOL/CPHA from a timing diagram and name the mode.
- Choose a bus for a stated device and pin budget.

Synchronous versus asynchronous

	UART (L10)	I2C	SPI
Clock line	none	SCL	SCK
Wires	2	2 + pull-ups	3 + one CS each
Addressing	none	7-bit, in protocol	physical (CS)
Acknowledgement	none	yes (ACK/NACK)	none
Typical speed	9.6–115 kbaud	100–400 kHz	1–50 MHz
Detects missing device	no	yes	no

The clock removes the rate assumption; the address removes the one-to-one restriction; the ACK removes the silence. SPI keeps the clock but drops the other two in exchange for speed.

I2C: the transaction

```
START | 7-bit addr + R/W | ACK | data byte | ACK | ... | STOP
|      |               |   |         |   |   |   |
master master      slave master      slave master
```

- Both lines are **open drain**: a device can only pull *low*. External **pull-ups** (4.7 kΩ) return them high. No pull-ups → lines stuck low → every transfer times out.
- Only the addressed device responds; all the others ignore the traffic.
- **ACK** = receiver pulled SDA low for one clock. **NACK** = left high.

The address shift

Form	Value	Used by
7-bit	0x27	datasheets, bus scanners, humans
8-bit	0x4E = 0x27 << 1	STM32 HAL

```
HAL_I2C_Master_Transmit(&hi2c1, (0x27 << 1), buf, n, 100); /* or 0x4E */
```

Passing 0x27 directly addresses device 0x13, which is not there — the single most expensive mistake on this bus. If a device will not answer, check the shift **before** the wiring.

I2C timing

Mode	Clock	Bit	Byte + ACK (9 bits)
Standard	100 kHz	10 μs	90 μs
Fast	400 kHz	2.5 μs	22.5 μs

The LCD backpack is a **port expander**: one character = **four** I2C bytes (two nibbles, each strobed with the enable line). So one character 45 bits **450 μ s** at 100 kHz, and a 16-character line **7.2 ms** — over seven tick periods. Refresh rate is a design decision with a measurable cost.

SPI: the four wires

Wire	Meaning
SCK	clock, from the master
MOSI	master out, slave in
MISO	master in, slave out
CS	chip select, active low, one per device

Every transfer is a **full-duplex exchange**: for each bit sent on MOSI, one arrives on MISO. To read, you must still write — usually a dummy byte.

```
HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_RESET); /* select */
HAL_SPI_TransmitReceive(&hspi2, tx, rx, n, 100);
HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_SET); /* deselect */
```

CS is an ordinary GPIO — HAL does not drive it. It must stay low for the whole transaction; releasing it between bytes resets most slaves.

CPOL, CPHA, and the four modes

Mode	CPOL	CPHA	Clock idles	Sampled on
0	0	0	low	leading (rising) edge
1	0	1	low	trailing (falling) edge
2	1	0	high	leading (falling) edge
3	1	1	high	trailing (rising) edge

Mode 0 is the most common. The **slave's** datasheet decides; the master obeys. A wrong mode does not give silence — it gives consistent, bit-shifted garbage, which is much harder to recognise.

Loopback

One jumper from MOSI to MISO makes the bus talk to itself:

```
uint8_t tx = 0xA5, rx = 0;
HAL_SPI_TransmitReceive(&hspi2, &tx, &rx, 1, 100); /* rx == 0xA5 */
```

This separates “my SPI is misconfigured” from “the slave is not answering,” and needs no slave. Remove the jumper and **rx** becomes 0x00 or 0xFF with **no error** — SPI cannot tell you a device is missing.

SFR evidence

Claim	Evidence
the I2C pins are open drain	GPIOB->OTYPER bits 8, 9 = 1
nothing answered	I2C1->SR1 AF (acknowledge failure) = 1
the bus is configured as I expect	I2C1->CCR for the clock rate
my SPI mode is what the slave wants	SPI2->CR1 CPOL/CPHA bits
the SPI link physically works	loopback: SPI2->DR returns what you wrote

Common mistakes

- The 7-bit/8-bit shift — 0x27 passed where 0x4E is wanted.
- No pull-ups: lines low, everything times out.
- SDA/SCL swapped — a bus scan then finds nothing at all.
- Ignoring the HAL return value; a NACK is real information.

- Wrong SPI mode: plausible but shifted data.
- CS released between bytes of one transaction.
- Expecting SPI to report a missing slave. It cannot.
- Refreshing the whole LCD every tick — 7.2 ms of I2C per line.

Mastery self-check

- ☐ I can name the three things a clock line and addressing buy over a UART.
- ☐ I can label START, address, R/W, ACK, and STOP on a trace.
- ☐ I can convert 7-bit 8-bit addresses in both directions.
- ☐ I can explain open drain and predict the failure without pull-ups.
- ☐ I can compute I2C byte and line times at 100 and 400 kHz.
- ☐ I can derive CPOL/CPHA from a timing diagram and name the mode.
- ☐ I can explain what loopback proves and what it does not.
- ☐ I can justify a bus choice from device count and pin budget.

Five review questions

1. A datasheet gives an address of 0x68. Give the value for HAL_I2C_Mem_Read, and explain what would be addressed if you passed 0x68.
2. Your bus scan finds nothing at all. List three distinct causes and the test that distinguishes each.
3. A 16-character line costs 7.2 ms at 100 kHz. You want the display and a 1 ms tick to coexist. Choose a refresh rate and defend it with numbers.
4. A slave's timing diagram shows the clock idling low and data sampled on the falling edge. Give CPOL, CPHA, and the mode. What do you see if you configure mode 0?
5. You must add eight identical sensors that each report once per second, and you have four spare pins. Choose a bus and justify it — then say what would change your answer.

See the L11 worksheet for extended practice. Worked solutions are released after the practice window.